Documentation

A guide to using LaTeX File Template v.4.0.0.

Matthew Emerson

August 2026

Contents

1 Introduction

Abstract

Thank you for choosing to use LaTeX File Template. If you find any bugs or want some feature not currently included, then my contact information can be found on my GitHub: https://github.com/MatthewEmer.

This template is intended for academics (with special focus on those in the fields of computer science or mathematics) who want to write papers or other related documents in a consistent format which ties in with standard practises from other published papers.

This guide consists of setup instructions to download the files and create your first document, before moving onto the custom commands/environments designed to speed up academic writing. Before using this guide, I would suggest that you have some practise with some basic LaTeX commands using a guide such as https://www.overleaf.com/learn.

This documentation is accurate as of Version 4.0.0.

Acknowledgements

I would like to thank Dr Jamie Mason for his support and expertise in leading my discrete mathematics tutorials and seminars this past year, without which this project would probably not exist. His continuous feedback throughout has allowed me to hone the project from the messy original versions into a useable tool.

Thanks also to Joseph Eddon for supporting the development of this template through the creation of his own LaTeX template, and for the feedback and suggestions he provided on multiple versions of the template throughout last year.

2 Document Setup

2.1 Project Files

The actual LaTeX file template is contained in the folder *The Template* and consists of five files: *template.tex*, *_documentContent.tex*, *_appendix.tex*, *_templateformatting.tex*, and *template.pdf*. To use the template, all four files must be downloaded and placed in the same folder.

2.1.1 _templateformatting.tex

This file contains the code required to create the template and the custom commands described in this document. It should not be edited.

2.1.2 _documentContent.tex

This file allows you to add all the main content of the document you are trying to write. It will be inserted after the table of contents, but before the appendix (if you choose to add one). By default it contains an example section and definition. Only pages in this file are counted towards the page count of the document.

2.1.3 _appendix.tex

This file allows you to add an optional appendix to the end of your document which is not counted towards your page count. By default it contains a unnumbered part title which has been inserted into the table of contents.

2.1.4 template.tex

This file sets up the structure of the document, as well as acting as a compiler for the other *.tex* files, ensuring they get inserted into the right sections of the document.

When creating your document, you need to edit lines 12-15 of this file to create the title page, alongside the document header and footer, as described in Table 1.

If you want to change the generated pdf's name while still writing the document, this is the file name which needs to be changed.

Line	Variable Name	Description
12	thetitle	The document's title. This will be placed in the header.
13	thesubtitle	The document's (optional) subtitle. This will be placed in the header.
14	theauthor	The list of authors. This will be placed in the footer.
15	thedate	Replace the code if you want a static date rather than date of last compile.

Table 1: Lines to edit.

2.1.5 template.pdf

This file contains the output of the LaTeX compiler. If you want to rename this document while still working on it, then delete this version, alongside any build files your compiler has created and change the name of *template.tex*.

2.2 Optional Files

2.2.1 Storing Images

If you choose to include figures in your project, then you will need to create a subfolder labelled *Images* where you can place the image files for the project.

2.2.2 GitHub Repositories

If you are placing the document within a GitHub repository, you may want to copy over the *.gitignore* from this repository as it is set up to ignore any LaTeX build files generated by LaTeX Workshop on Visual Studio Code. These are: $*.aux$, $*.fdb\_latexmk$, $*.fls$, $*.log$, $*.out$, $*.gx$, $*.toc$, and $*.gz$.

2.3 Updating the Template

If you have downloaded the template and a new version is released, then you can easily update your document to the new version by taking the *_templateformatting.tex* file currently in your project and replacing it the one from the new release. This is generally not advised for large releases (where the first digit of the version number changes) as other files may have been changed.

3 Custom Commands

3.1 General Tools

3.1.1 Element Spacing

Command 3.1.1.1: *\separate*

The separate command inserts vertical spacing between two elements whilst simultaneously removing the new paragraph indent from the following element. You can specify the amount of spacing as described in Table 2 and as seen in Example 3.1.1.1.

No.	Name	Description	Example Value
1	Spacing	The number of points of vertical spacing.	5

Table 2: Parameters for separate.

Example 3.1.1.1: *\separate*{3} produces the separation as seen between the Command line and the description of separate.

3.1.2 Text Formatting

Command 3.1.2.1: *\colouredText*

The colouredText command takes the given text and outputs it in a different colour, as chosen by the writer. This can be seen in Example 3.1.2.1 and as described in Table 3.

No.	Name	Description	Example Value
1	Colour	The colour to change the text to.	blue
2	Text	The text you want outputted in colour.	Hello World

Table 3: Parameters for coloured Text.

Example 3.1.2.1: *\colouredText*{*blue*}{Hello World} produces Hello World.

Command 3.1.2.2: *\comment*

The comment command inserts notes into the document in red so they can be found more easily. This can be seen in Example 3.1.2.2 and as described in Table 4.

No.	Name	Description	Example Value
1	Text	The text you want outputted in colour.	Hello World

Table 4: Parameters for comment.

Example 3.1.2.2: *\comment*{Hello World} produces Hello World.

3.1.3 Date Formatting

Command 3.1.3.1: *\yeardate*

The yeardate command takes an adjacent date variable and reformats it into the four-digit year before outputting to the page as seen in Example 3.1.3.1.

No.	Name	Description	Example Value
	n/a		

Table 5: Parameters for yeardate.

Example 3.1.3.1: *\date{\yeardate\today}* produces 2026 (at time of writing).

Command 3.1.3.2: *\monthyeardate*

The monthyeardate command takes an adjacent date variable and reformats it into the full month and four-digit year before outputting to the page as seen in Example 3.1.3.2.

No.	Name	Description	Example Value
	n/a		

Table 6: Parameters for monthyeardate.

Example 3.1.3.2: *\date{\monthyeardate\today}* produces August 2026 (at time of writing).

3.2 Tables and Figures

3.2.1 Inserting Tables

Command 3.2.1.1: *\inserttable*

The inserttable command simplifies the process of inserting tables into your document as described in Table 7 and seen in Example 3.2.1.1.

No.	Name	Description	Example Value
1	Columns	The required columns, separated by |s.	*c|c|c|c*
2	Caption	The caption to appear under the table.	Parameters for inserting tables.
3	Headers	The headers, separated by ampersands.	No. & Name & ...
4	Content	The content to be placed in the table.	

Table 7: Parameters for inserttable.

Example 3.2.1.1: *\inserttable{c|c|c|c}*{Parameters...}{No. & Name & ...}{1 & Columns & ...} produces Table 7.

3.2.2 Inserting Figures

Command 3.2.2.1: *\insertimage*

The insertimage command simplifies the process of inserting figures into your document as described in Table 8 and seen in Example 3.2.2.1.

No.	Name	Description	Example Value
1	File name	The file name of the image (must be in *Images*.)	durham.jpg
2	Figure caption	The caption to appear under the figure.	Durham Cathedral.
3	Width	The amount of the page width the image should occupy.	0.3

Table 8: Parameters for insertimage.

Example 3.2.2.1: *\insertimage{durham.jpg}*{A large building sitting on top of a lush green hillside. (Stoll, 2022)}{0.3} produces Figure 1.

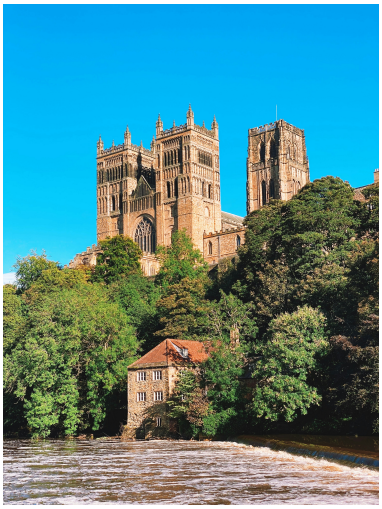

Figure 1: A large building sitting on top of a lush green hillside. (Stoll, 2022)

3.2.3 Referencing Tables and Figures

Command 3.2.3.1: *\elemref*

The elemref command allows you to easily reference tables and figures with clickable links. This is described in Table 9 and as seen in Example 3.2.3.1.

No.	Name	Description	Example Value
1	Label	The label of the element being linked to.	tab:Parameters for elemref.
2	Type	The type of element being linked to.	Table

Table 9: Parameters for elemref.

Example 3.2.3.1: *\elemref*{tab:Parameters for elemref.}{Table} produces the reference Table 9.

3.3 Academic Environments

3.3.1 Basic Environments

Command 3.3.1.1: *\definition*

The definition command creates an environment for setting out a definition with automatic numbering with inbuilt labels so that you can reference the definition later in the document. Its parameters can be seen in Table 10 and an example of its use can be seen in Example 3.3.1.1.

No.	Name	Description	Example Value
1	Label	The word being defined so it can be easily referenced.	example
2	Definition	The actual definition of the word (with the word underlined).	An example is...

Table 10: Parameters for definition.

Example 3.3.1.1: *\definition*{example}{An *\underline*{example} is what this is.} produces Definition 3.3.1.1

Definition 3.3.1.1: An example is what this is.

Command 3.3.1.2: *\conjecture*

The conjecture command creates an environment for setting out a conjecture with automatic numbering with inbuilt labels so that you can reference the conjecture later in the document. Its parameters can be seen in Table 11 and an example of its use can be seen in Example 3.3.1.2.

No.	Name	Description	Example Value
1	Label	The conjecture's name.	Someone's Conjecture
2	Conjecture	The content for the conjecture.	Someone's Conjecture states...

Table 11: Parameters for conjecture.

Example 3.3.1.2: *\conjecture*{Someone's Conjecture}{Someone's Conjecture states something or other.} produces Conjecture 3.3.1.1

Conjecture 3.3.1.1: Someone's Conjecture states something or other.

3.3.2 Environments with Proofs or Solutions

Command 3.3.2.1: *\theorem*

The theorem command creates an environment for setting out a theorem with automatic numbering with inbuilt labels so that you can reference the theorem later in the document. Its parameters can be seen in Table 12 and an example of its use can be seen in Example 3.3.2.1.

No.	Name	Description	Example Value
1	Label	The theorem's name.	Example Theorem
2	Definition	The theorem definition.	If $2x = y$ then $8x = 4y$.
3	Proof	The proof of the theorem.	If $2x = y$...

Table 12: Parameters for theorem.

Example 3.3.2.1: *\theorem*{Example Theorem}{If $2x = y$ then...}{If $2x = y$, we...} produces Theorem 3.3.2.1

Theorem 3.3.2.1: If $2x = y$ then $8x = 4y$ holds for all $(x, y) \in \mathbb{N}^2$.

Proof: If $2x = y$, we can factor $8x = 4y$ by taking out a common multiplier of 4 to get $4(2x = y)$. Therefore, the theorem must hold for all $(x, y) \in \mathbb{N}^2$.

□

Command 3.3.2.2: *\lemma*

The lemma command creates an environment for setting out a lemma with automatic numbering with inbuilt labels so that you can reference the lemma later in the document. Its parameters can be seen in Table 13 and an example of its use can be seen in Example 3.3.2.2.

No.	Name	Description	Example Value
1	Label	The lemma's name.	Example Lemma
2	Definition	The lemma definition.	If $2x = y$ then $8x = 4y$.
3	Proof	The proof of the lemma.	If $2x = y$...

Table 13: Parameters for lemma.

Example 3.3.2.2: *\lemma*{Example Lemma}{If $2x = y$ then...}{If $2x = y$, we...} produces Lemma 3.3.2.1

Lemma 3.3.2.1: If $2x = y$ then $8x = 4y$ holds for all $(x, y) \in \mathbb{N}^2$.

Proof: If $2x = y$, we can factor $8x = 4y$ by taking out a common multiplier of 4 to get $4(2x = y)$. Therefore, the lemma must hold for all $(x, y) \in \mathbb{N}^2$.

□

Command 3.3.2.3: *\proposition*

The proposition command creates an environment for setting out a proposition with automatic numbering with inbuilt labels so that you can reference the proposition later in the document. Its parameters can be seen in Table 14 and an example of its use can be seen in Example 3.3.2.3.

No.	Name	Description	Example Value
1	Label	The proposition's name.	Example Proposition
2	Definition	The proposition definition.	If $2x = y$ then $8x = 4y$.
3	Proof	The proof of the proposition.	If $2x = y$...

Table 14: Parameters for proposition.

Example 3.3.2.3: *\proposition*{Example Proposition}{If $2x = y$ then...}{If $2x = y$, we...} creates Proposition 3.3.2.1

Proposition 3.3.2.1: If $2x = y$ then $8x = 4y$ holds for all $(x, y) \in \mathbb{N}^2$.

Proof: If $2x = y$, we can factor $8x = 4y$ by taking out a common multiplier of 4 to get $4(2x = y)$. Therefore, the lemma must hold for all $(x, y) \in \mathbb{N}^2$.

□

Command 3.3.2.4: *\standardproblem*

The standardproblem command creates an environment for setting out a standard problem with inbuilt labels so that you can reference the standard problem later in the document. Its parameters can be seen in Table 15 and an example of its use can be seen in Example 3.3.2.4. When referencing a standard problem, the native *\hyperref* command should be used.

No.	Name	Description	Example Value
1	Number	The problem number.	1
2	Problem	The problem statement.	If $2x = 4$, what is x?
3	Solution	The problem's solution.	$2x = 4$, divide...

Table 15: Parameters for standardproblem.

Example 3.3.2.4: *\standardproblem*{1}{If $2x = 4$ what is x}{$2x = 4$...} creates Standard Problem #1.

Standard Problem #1: If $2x = 4$, what is x?

Solution: $2x = 4$, divide both sides by 2, $x = 2$.

□

3.3.3 Sub-Environments

Important Note: Both of the following commands can only be referenced using *\elemref* as opposed to the environment references used by the rest of this section.

Command 3.3.3.1: *\proof*

The proof command creates an environment for setting out a proof with inbuilt labels so that you can reference the proof later in the document. Its parameters can be seen in Table 16 and an example of its use can be seen in Example 3.3.3.1.

No.	Name	Description	Example Value
1	Label	The proof's name.	Example Proof
2	Proof	The content for the proof.	Let $x = ...$

Table 16: Parameters for proof.

Example 3.3.3.1: *\proof*{prop:Example Proposition}{If $2x = y$...} produces Proof 3.3.2.

Command 3.3.3.2: *\solution*

The solution command creates an environment for setting out a solution with inbuilt labels so that you can reference the solution later in the document. Its parameters can be seen in Table 17 and an example of its use can be seen in Example 3.3.3.2.

No.	Name	Description	Example Value
1	Label	The solution's name.	Example Solution
2	Solution	The content for the solution.	Let $x = ...$

Table 17: Parameters for solution.

Example 3.3.3.2: *\solution*{1}{$2x = 4$, divide...} produces Solution 3.3.2.

3.3.4 Customisable Environments

Command 3.3.4.1: *\numberedentry*

The numberedentry command creates an environment for setting out a custom environment with automatic numbering with inbuilt labels so that you can reference it later in the document. Its parameters can be seen in Table 18 and an example of its use can be seen in Example 3.3.4.1.

No.	Name	Description	Example Value
1	Label	The entry's name.	cmd:numberedentry
2	Entry Type	The type of entry.	Command
3	Content	The content of the entry.	The numberedentry command...

Table 18: Parameters for numberedentry.

Example 3.3.4.1: *\numberedentry* produces Command 3.3.4.1.

3.3.5 Referencing Environments

Command 3.3.5.1: *\preenvref*

The preenvref command allows you to easily reference academic environments with clickable links before they are defined in the document. This is described in Table 19 and as seen in Example 3.3.5.1.

No.	Name	Description	Example Value
1	Label	The label of the environment being linked to.	ex:cmd:postenvref
2	Type	The type of environment being linked to.	Example
3	Counter	The name of the counter for that environment.	examplenumber

Table 19: Parameters for preenvref and postenvref.

Example 3.3.5.1: *\preenvref*{ex:cmd:preenvref}{Example}{examplenumber} produces the reference Example 3.3.5.1.

Command 3.3.5.2: *\postenvref*

The postenvref command allows you to easily reference academic environments with clickable links after they are defined in the document. This is described in Table 19 and as seen in Example 3.3.5.2.

Example 3.3.5.2: *\postenvref*{ex:cmd:preenvref}{Example}{examplenumber} produces the reference Example 3.3.5.1.

3.4 Mathematics

3.4.1 Basic Mathematics Notation

Command 3.4.1.1: Insert standard number set symbols.

This set of commands allows you to quickly insert the set symbols for all of the common number sets used in mathematics. The supported sets, and their associated commands, can be seen in Table 20 below.

Number Set	Command	Output
Natural	$\N$	$\mathbb{N}$
Integers	$\Z$	$\mathbb{Z}$
Rational	$\Q$	$\mathbb{Q}$
Real	$\R$	$\mathbb{R}$
Imaginary	$\I$	$\mathbb{I}$
Complex	$\C$	$\mathbb{C}$

Table 20: Supported number sets.

3.4.2 Vectors and Matrices

Command 3.4.2.1: *\dotproduct*

The dotproduct command allows you to easily use the dot product vector operation within equations. The parameters for the function are described in Table 21, and an example can be seen Example 3.4.2.1.

No.	Name	Description	Example Value
1	Vector A	The vector on the left of the operation.	u
2	Vector B	The vector on the right of the operation	v

Table 21: Parameters for dotproduct.

Example 3.4.2.1: *\dotproduct*{u}{v} produces $\langle \mathbf{u}, \mathbf{v} \rangle$.

Command 3.4.2.2: *\jacobian*

The Jacobian command inserts the notation for the Jacobian matrix of a given function. This is described in Table 22, and can be seen in Example 3.4.2.2.

No.	Name	Description	Example Value
1	Function	The function the Jacobian is generated from.	f

Table 22: Parameters for jacobian.

Example 3.4.2.2: *\jacobian*{f} produces $\mathbf{J}_f$.

Command 3.4.2.3: $\backslash hessian$

The Hessian command inserts the notation for the Hessian matrix of a given function. This is described in Table 23, and can be seen in Example 3.4.2.3.

No.	Name	Description	Example Value
1	Function	The function the Hessian is generated from.	f

Table 23: Parameters for hessian.

Example 3.4.2.3: $\backslash hessian\{f\}$ produces $\mathbf{H}_f$.

Command 3.4.2.4: $\backslash lagrangian$

The Lagrangian command inserts the notation for a Lagrangian function. This is described in Table 24, and can be seen in Example 3.4.2.4.

No.	Name	Description	Example Value
	n/a		

Table 24: Parameters for lagrangian.

Example 3.4.2.4: $\backslash lagrangian$ produces $\mathcal{L}$.

3.5 Computer Science

3.5.1 Relational Algebra

Command 3.5.1.1: Insert row join symbols.

This set of commands allows you to quickly insert the symbols for all of the common join types used in relational algebra. The supported joins, and their associated commands, can be seen in Table 25 below.

Number Set	Command	Output
Inner Join	$\backslash innerjoin$	$\bowtie$
Left Outer Join	$\backslash leftouterjoin$	$⟕$
Right Outer Join	$\backslash rightouterjoin$	$⟖$
Full Outer Join	$\backslash fullouterjoin$	$⟗$

Table 25: Supported joins.

3.5.2 Algorithms

Command 3.5.2.1: $\backslash insertalgorithm$

The insertalgorithm command inserts a box suited for a pseudocode algorithm to be inserted into the document. This is described in Table 26, and can be seen in Example 3.5.2.1.

No.	Name	Description	Example Value
1	Algorithm Name	The name of the algorithm.	Hello or goodbye
2	Inputs	The required inputs.	n/a
3	Outputs	Any returned values.	n/a
4	Pseudocode	The pseudocode of the algorithm.	$\backslash Procedure$\{helloOrGoodbye\}\{\}...

Table 26: Parameters for insertalgorithm.

Example 3.5.2.1: *insertalgorithm*{Hello or goodbye}{n/a}{n/a}{*Procedure*{helloOrGoodbye}{}...} produces Algorithm 1.

Algorithm 1 Hello or goodbye

Input: $x \in \{1, 2\}$
Output: A string message.

procedure HelloOrGoodbye(x)
    if (x == 1) **then return** "Hello World"
    else if (x == 2) **then return** "Goodbye"
    else return "418 I am a teapot."
    end if
end procedure

4 Appendix

4.1 List of Figures

Figure 1

Stoll, M. (2022). A large building sitting on top of a lush green hillside. [Online] unsplash.com. Available at: https://unsplash.com/photos/a-large-building-sitting-on-top-of-a-lush-green-hillside-jATW0-hO4-I [Accessed 10 Aug. 2026].